

SKILL9.10.11.12

9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic
INPU

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <limits.h>

int main() {
    char input[200];
    char previous[PATH_MAX] = "";

    while (1) {
        char current[PATH_MAX];

        getcwd(current, sizeof(current));
        printf("%s$ ", current);

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        // Exit
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // cd command
        if (strncmp(input, "cd ", 3) == 0) {
            char *path = input + 3;

            // Save current directory
            strcpy(previous, current);

            if (chdir(path) == -1) {
                perror("cd failed");
            } else {
                printf("Directory changed successfully.\n");
            }
        }
    }
}
```

```

// cd -
else if (strcmp(input, "cd -") == 0) {
    if (strlen(previous) == 0) {
        printf("Previous directory not available.\n");
    } else {
        char temp[PATH_MAX];

        strcpy(temp, current);

        if (chdir(previous) == -1) {
            perror("cd failed");
        } else {
            strcpy(previous, temp);
            printf("Changed to previous directory.\n");
        }
    }
}

else {
    printf("Unknown command: %s\n", input);
}

return 0;

```

OUTPUT:

```

sushmitha@LAPTOP-B907B06L:~$ nano skill9.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill9.c -o skill9
sushmitha@LAPTOP-B907B06L:~$ ./skill9
/home/sushmitha$ cd /tmp
Directory changed successfully.

```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

INPUT:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char input[200];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        // pwd command
        if (strcmp(input, "pwd") == 0) {
            char cwd[PATH_MAX];

            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("%s\n", cwd);
            else
                perror("getcwd");
        }

        // export command
        else if (strncmp(input, "export ", 7) == 0) {
            char *variable = input + 7;

            if (putenv(variable) == 0)
                printf("Variable exported successfully.\n");
            else
                perror("export");
        }

        // exit command

```

1.

```

    else if (strcmp(input, "exit") == 0) {
        printf("Exiting shell...\n");
        break;
    }

    else if (strlen(input) == 0) {
        continue;
    }

    else {
        printf("Unknown command: %s\n", input);
    }
}

return 0;

```

OUTPUT:

```

sushmitha@LAPTOP-B907B06L:~$ nano skill10.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill10.c -o skill10
sushmitha@LAPTOP-B907B06L:~$ ./skill10
myshell> export NAME = sony
Variable exported successfully.

```

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

2. INPUT:

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_CMD 100

int main() {
    char history[MAX_HISTORY][MAX_CMD];
    char command[MAX_CMD];
    int count = 0;

    printf("Simple Command History\n");
    printf("Type 'history' to display commands\n");
    printf("Type 'exit' to quit\n\n");

    while (1) {
        printf("shell> ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d %s\n", i + 1, history[i]);
            }

            continue;
        }

        if (strlen(command) == 0)
            continue;
    }
}
```

```

        continue;
    }

    if (strlen(command) == 0)
        continue;

    if (count < MAX_HISTORY) {
        strcpy(history[count], command);
        count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
            strcpy(history[i], history[i + 1]);

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

return 0;
}

```

OUTPUT:

```

sushmitha@LAPTOP-B907B06L:~$ nano skill11.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill11.c -o skill11
sushmitha@LAPTOP-B907B06L:~$ ./skill11
Simple Command History
Type 'history' to display commands
Type 'exit' to quit

shell> ls
shell> pwd
shell> date
shell> history

Command History:
1  ls
2  pwd
3  date
shell> |

```

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Input:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);
```

```
    execlp("wc", "wc", "-l", NULL);

    perror("execlp");
    exit(1);
}

close(fd[0]);
close(fd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

return 0;
}
```

Output:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill12.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill12.c -o skill12
sushmitha@LAPTOP-B907B06L:~$ ./skill12
60
```